SISTEMA DE CONTROL Y PLANIFICACIÓN DE DRONES PARA LA TRANSMISIÓN ÓPTIMA DE MENSAJES UTILIZANDO ESTRUCTURAS DE DATOS DINÁMICAS

202300583 – Angel Sebastian Mazariegos Pérez

Resumen

Este proyecto desarrolla un sistema web para la gestión y emisión de mensajes mediante drones, implementado en ASP.NET Core con arquitectura MVC. La solución integra estructuras de datos dinámicas creadas manualmente, como listas enlazadas, pilas y colas, para administrar drones, sistemas y mensajes sin usar colecciones nativas. Además, incorpora carga incremental de archivos XML, validación de configuraciones, planificación de instrucciones segundo a segundo y generación de salidas en XML. El sistema también permite visualizar gráficamente los sistemas de drones y las secuencias de ejecución mediante Graphviz, facilitando la comprensión del proceso de optimización. Entre sus aportes principales destaca el soporte para múltiples mensajes, el reconocimiento correcto de espacios en blanco dentro del texto y la reconstrucción del mensaje a partir de la configuración del sistema. En conjunto, la aplicación resuelve de forma ordenada la coordinación de drones, la representación de mensajes codificados y la exportación de resultados, cumpliendo con las restricciones del proyecto y ofreciendo una base funcional para futuras ampliaciones.

Palabras clave

emisor de drones, ASP.NET Core, XML, Graphviz, estructuras de datos

Abstract

This project develops a web-based system for drone-based message emission using ASP.NET Core and the MVC architecture. The solution integrates manually implemented dynamic data structures, such as linked lists, stacks, and queues, to manage drones, systems, and messages without relying on native collections. It also includes incremental XML file loading, configuration validation, second-by-second instruction planning, and XML output generation. In addition, the system provides graphical visualization of drone systems and execution timelines through Graphviz, making the optimization process easier to understand. Its main contributions include support for multiple messages, correct handling of whitespace inside text, and message reconstruction from the system configuration. Overall, the application organizes drone coordination, encoded message representation, and result export in a practical way, meeting the project constraints while providing a solid foundation for future improvements.

Keywords

drone emitter, ASP.NET Core, XML, Graphviz, data structures

Figura 7. Diagrama de clases del sistema
Fuente: realización propia

Introducción

El proyecto Emisor de Drones desarrolla una solución web para coordinar drones capaces de transmitir mensajes mediante la emisión de luz. Su importancia radica en que integra procesamiento de datos, validación de estructuras XML y planificación de instrucciones segundo a segundo para representar mensajes codificados de forma ordenada y eficiente. Además, el sistema utiliza estructuras de datos implementadas manualmente, como listas enlazadas, pilas y colas, lo que refuerza el enfoque académico y el control sobre la lógica interna del programa. A través de una interfaz MVC, el usuario puede cargar archivos, visualizar sistemas de drones, revisar mensajes procesados y generar salidas en XML. Este trabajo permite comprender cómo se organiza la información, cómo se optimiza la ejecución de instrucciones y cómo se representa visualmente el comportamiento del sistema.

Desarrollo del tema

1. **Gestión y carga de información**
   El sistema Emisor de Drones permite cargar información estructurada en formato XML para registrar drones, sistemas de drones y mensajes. Esta etapa es importante porque organiza los datos de entrada y valida que la información sea consistente antes de procesarla. La aplicación también soporta carga incremental, lo que facilita incorporar nuevos datos sin perder la información ya almacenada en memoria. Esta parte del proyecto resuelve uno de los problemas centrales del manejo de configuraciones: mantener orden, validación y continuidad en los datos.

Figura 1. Carga de archivo XML de entrada.

Fuente: elaboración propia

2. **Estructuras de datos implementadas**
   El proyecto utiliza estructuras de datos creadas manualmente, como lista enlazada, pila y cola, para administrar la información sin depender de colecciones nativas. Esto permite controlar mejor el comportamiento interno del sistema y cumple con las restricciones académicas del proyecto. La lista enlazada se usa para almacenar drones, sistemas y mensajes; la pila y la cola apoyan procesos internos de organización y recorrido. Este enfoque fortalece la comprensión de la memoria dinámica y el manejo de nodos enlazados.

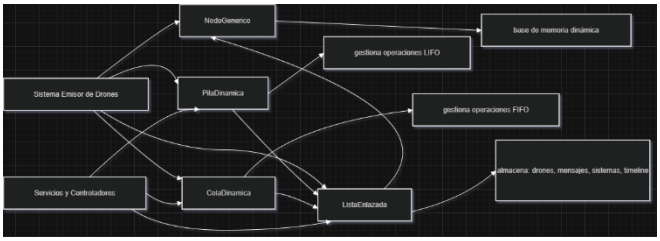


Figura 2. Estructuras de datos utilizadas en el sistema.

Fuente: elaboración propia

3. **Planificación y optimización de mensajes**
   Una vez cargada la información, el sistema construye una secuencia optimizada de instrucciones para que los drones emitan los símbolos correctos en el momento adecuado. El motor de optimización genera una línea de tiempo segundo a segundo, respetando la restricción de que solo un dron puede emitir luz por segundo. Además, el sistema reconstruye el texto del

mensaje a partir de las alturas y símbolos configurados en el XML, incluyendo espacios en blanco cuando corresponden. Esta parte demuestra cómo el programa transforma datos codificados en un mensaje legible.

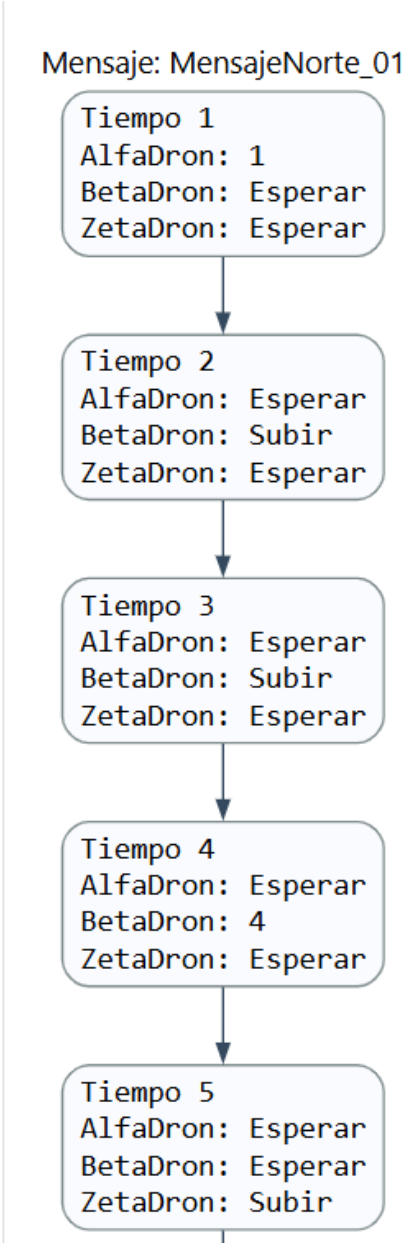
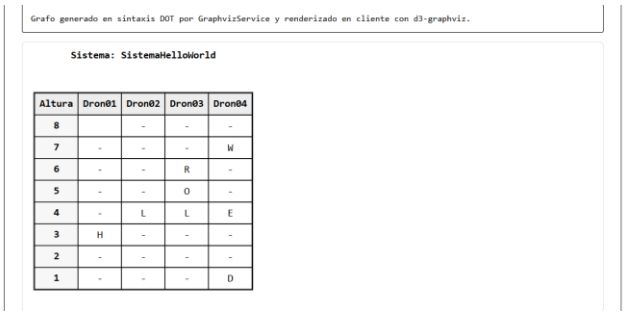


Figura 3. Secuencia optimizada de ejecución del mensaje.

Fuente: elaboración propia

4. Visualización y salida de resultados

El sistema incluye vistas gráficas para revisar los sistemas de drones y las instrucciones calculadas, usando Graphviz para representar la información de forma clara. También genera una salida XML con el resultado del procesamiento, lo cual permite verificar el comportamiento del programa y conservar la información final. Esta integración entre visualización, cálculo y exportación muestra que la solución no solo procesa datos, sino que también los presenta y documenta adecuadamente.



Altura	Dron01	Dron02	Dron03	Dron04
8		-	-	-
7	-	-	-	W
6	-	-	R	-
5	-	-	O	-
4	-	L	L	E
3	H	-	-	-
2	-	-	-	-
1	-	-	-	D

Figura 4. Visualización del sistema de drones.

Fuente: elaboración propia

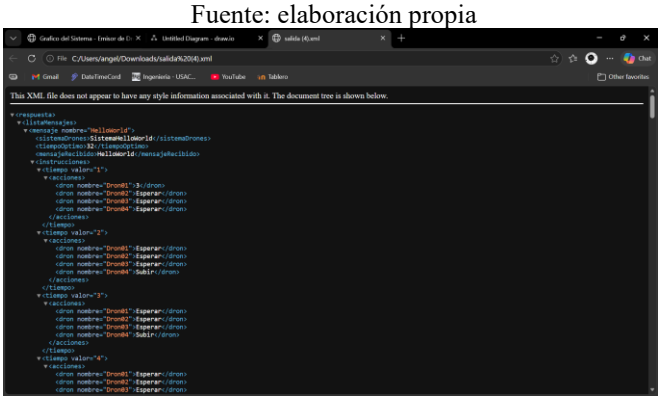


Figura 5. Archivo XML generado con los resultados.

Fuente: elaboración propia

Tabla I. Componentes principales del sistema

Módulo	Función

Carga XML	Lee y valida la información de entrada
Servicio de drones	Administra el registro y eliminación de drones
Motor de optimización	Genera la secuencia de instrucciones
Visualización Graphviz	Representa sistemas y tiempos de ejecución
XML de salida	Exporta el resultado final del procesamiento

Fuente: elaboración propia.

La Tabla I resume la función técnica de cada componente principal del sistema y su aporte dentro del flujo general: carga de datos, validación, planificación, visualización y generación de salida. Más que listar módulos, la tabla permite identificar cómo se conectan entre sí para transformar un XML de entrada en un mensaje procesado y verificable. De esta manera, se evidencia que la solución no depende de un único bloque, sino de la integración coordinada de estructuras de datos, servicios de negocio y vistas web.

Conclusiones

A partir del desarrollo realizado, se concluye que el proyecto Emisor de Drones cumple su objetivo principal al integrar carga, validación, planificación, visualización y exportación de información en una sola solución funcional. El uso de estructuras de datos implementadas manualmente demostró que es posible construir una arquitectura sólida sin depender de colecciones nativas, fortaleciendo tanto la comprensión técnica como la consistencia del sistema. Asimismo, la incorporación de mejoras como la carga incremental, la reconstrucción del texto del mensaje y el manejo correcto de espacios en blanco evidencian un avance significativo en precisión y robustez del procesamiento.

Como postura principal, se sostiene que la calidad de una solución no depende solo de que “funcione”, sino de su capacidad para validar datos, representar resultados con claridad y mantenerse extensible ante nuevos requerimientos. En ese sentido, este trabajo aporta una base confiable para futuras ampliaciones, por ejemplo, optimizaciones más avanzadas, persistencia en base de datos o simulación en tiempo real. Finalmente, queda abierta la reflexión sobre cómo equilibrar complejidad algorítmica, legibilidad del código y experiencia del usuario en sistemas técnicos de coordinación automatizada.

Referencias bibliográficas

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C., (2009). Introduction to Algorithms (3rd ed.). The MIT Press.

Freeman, A., (2021). Pro ASP.NET Core 6 (9th ed.). Apress.

Microsoft, (2024). ASP.NET Core documentation. Microsoft
Learn. https://learn.microsoft.com/aspnet/core

Skeet, J., (2019). C# in Depth (4th ed.). Manning Publications.

W3C, (2008). Extensible Markup Language (XML) 1.0 (Fifth Edition). World Wide Web Consortium. https://www.w3.org/TR/xml/

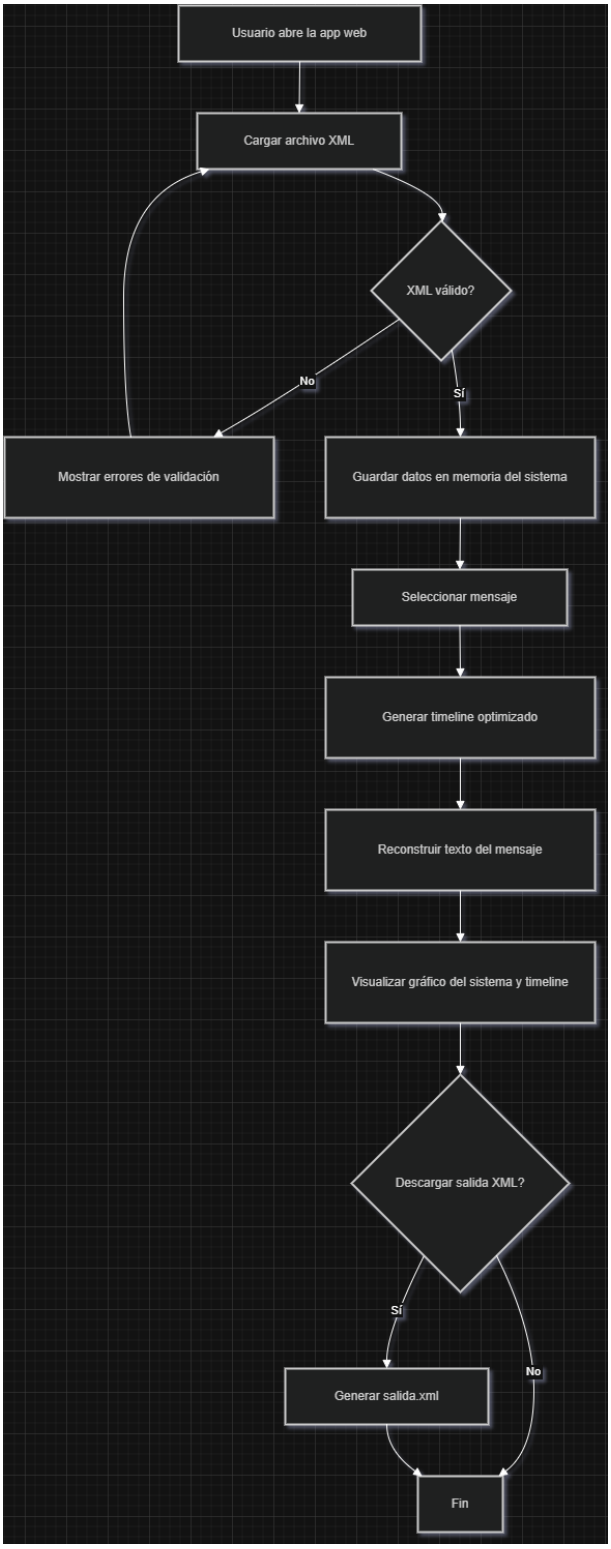
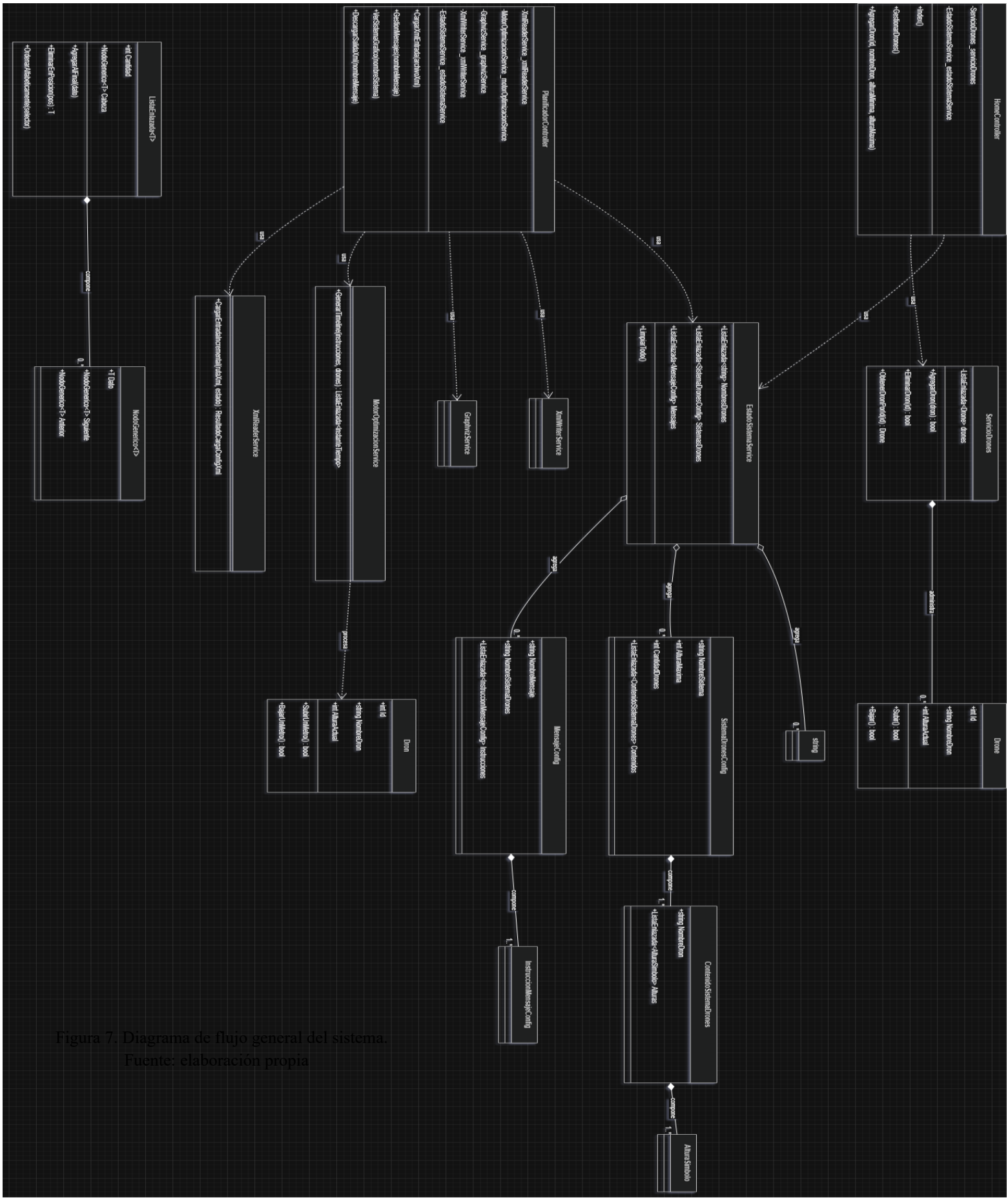


Figura 6. Diagrama de flujo general del sistema.
Fuente: elaboración propia


HomeController
-ServicioDrones _servicioDrones
-EstadoSistemaService _estadoSistemaService
+Index()
+GestionarDrones()
+AgregarDrone(id, nombreDron, alturaMinima, alturaMaxima)
ServicioDrones
-ListaEnlazada<Drone> drones
+AgregarDron(dron) : bool
+EliminarDron(id) : bool
+ObtenerDronPorId(id) : Drone
Drone
+int Id
+string NombreDron
+int AlturaActual
+Subir() : bool
+Bajar() : bool
EstadoSistemaService
+ListaEnlazada<string> NombresDrones
+ListaEnlazada<SistemaDronesConfig> SistemasDrones
+ListaEnlazada<MensajeConfig> Mensajes
+LimpiarTodo()
SistemaDronesConfig
+string NombreSistema
+int AlturaMaxima
+int CantidadDrones
+ListaEnlazada<ContenidoSistemaDrones> Contenidos
ContenidoSistemaDrones
+string NombreDron
+ListaEnlazada<AlturaSimbolo> Alturas
AlturaSimbolo
MensajeConfig
+string NombreMensaje
+string NombreSistemaDrones
+ListaEnlazada<InstruccionMensajeConfig> Instrucciones
InstruccionMensajeConfig
string
PlanificadorController
-XmlReaderService _xmlReaderService
-MotorOptimizacionService _motorOptimizacionService
-GraphvizService _graphvizService
-XmlWriterService _xmlWriterService
-EstadoSistemaService _estadoSistemaService
+CargarXmlEntrada(archivoXml)
+GestionMensajes(nombreMensaje)
+VerSistemaGrafico(nombreSistema)
+DescargarSalidaXml(nombreMensaje)
XmlWriterService
GraphvizService
MotorOptimizacionService
+GenerarTimeline(instrucciones, drones) : ListaEnlazada<InstanteTiempo>
XmlReaderService
+CargarEntradaIncremental(rutaXml, estado) : ResultadoCargaConfigXml
Dron
+int Id
+string NombreDron
+int AlturaActual
+SubirUnMetro() : bool
+BajarUnMetro() : bool
ListaEnlazada<T>
+int Cantidad
+NodoGeneric<T> Cabeza
+AgregarAlFinal(dato)
+EliminarEnPosicion(pos) : T
+ObtenerNodoEnPosicion(pos)
NodoGeneric<T>
+T Dato
+NodoGeneric<T> Siguiente
+NodoGeneric<T> Anterior
usa
usa
usa